Diseño e Implementación de una Máquina para Encriptar Mensajes Usando Números Primos Grandes

Santiago Alejandro Céspedes Daza
Facultad de Ingeniería, Universidad Sergio Arboleda
Sistemas Complejos
Bogotá, Colombia

***Resumen*—** Se diseñó e implementó una máquina parametrizable que, dados una técnica, una semilla y un número de cifras, produce números pseudoaleatorios de 40 cifras. La máquina incluye cuatro técnicas: generador congruencial lineal (LCG), Mersenne Twister, SHA-256 en modo contador y Blum-Blum-Shub. Con cada técnica se generaron 10 000 números de 40 cifras y se identificaron los primos mediante Miller-Rabin/Baillie-PSW (verificados con una segunda implementación), obteniendo 87 a 116 primos por técnica frente a 109.3 esperados por el Teorema de los Números Primos. Con los primos hallados se construyó un sistema de encriptación RSA por bloques capaz de cifrar mensajes de longitud arbitraria (UTF-8) y números, y se ejecutó una batería de 25 pruebas de encriptación, superadas en su totalidad. La máquina se puede usar por línea de comandos, mediante un menú interactivo o con una interfaz gráfica. Se documentan además una falla real del LCG con módulo potencia de 2 y las limitaciones de seguridad del sistema.

***Palabras clave*—**Números pseudoaleatorios, PRNG, números primos, RSA, criptografía, Blum-Blum-Shub, Miller-Rabin.

I. LA MÁQUINA

La máquina, implementada en la clase `MaquinaPRNG`, recibe tres parámetros y entrega números de exactamente `cifras` dígitos:

Cuadro 1. Parámetros de entrada de la máquina

Parámetro	Significado
`tecnica`	`lcg`, `mt`, `sha256` o `bbs`
`semilla`	entero; misma semilla ⇒ misma salida
`cifras`	dígitos de la salida (por defecto 40)
`cantidad`	cuántos números emitir (CLI / `lote`)

Cada técnica entrega un entero crudo de $\lceil \log_2(\text{rango}) \rceil + 64$ bits; la máquina lo reduce al intervalo $[10^{c-1}, 10^{c})$ con un módulo. El margen de 64 bits hace que el sesgo de la reducción sea menor que 2^{-64}. Ejemplo de uso:

```
$ python generadores_primos.py maquina \
    --tecnica sha256 --semilla 42 --cifras 40
7968969036072618282685701201480857869219
```

A. Las cuatro técnicas

LCG (Lehmer/Park-Miller [2]): $X_{n+1} = aX_n$ mód $(2^{61} - 1)$, con bloques de 61 bits concatenados. **Mersenne Twister** MT19937: periodo $2^{19937} - 1$, no criptográfico (estado recuperable tras 624 salidas). **SHA-256 en modo contador**: $H = \text{SHA256}(\text{semilla} \,\|\, \text{contador})$; es un CSPRNG. **Blum-Blum-Shub** [3]: $x_{k+1} = x_k^2$ mód pq con $p \equiv q \equiv 3 \pmod 4$ (aquí de 128 bits), emitiendo la paridad de cada x_k; su seguridad se reduce a la dificultad de factorizar pq.

B. Hallazgo: la falla del LCG con módulo 2^{64}

La primera versión del LCG usó los parámetros clásicos de Knuth con módulo 2^{64} y produjo **0 primos en 100 000 números**: con módulo potencia de 2 y a, c impares, el bit menos significativo alterna $0, 1, 0, 1, \ldots$ y todos los números resultan pares [1]. Se corrigió con módulo primo $(2^{61} - 1)$. Hoy el LCG produce 49.8 % de pares, como se espera.

II. EXPERIMENTO: 10 000 NÚMEROS DE 40 CIFRAS POR TÉCNICA

Con semilla 2026 se generaron 10 000 números por técnica (40 000 en total). Se verificó que todos tuvieran exactamente 40 cifras y que no hubiera repetidos (0 en las cuatro técnicas). La primalidad se evaluó con `gmpy2.is_prime` (Miller-Rabin + Baillie-PSW). Como control independiente, *todos* los primos hallados y 600 compuestos por técnica se revisaron con `sympy.isprime`: coincidieron en el 100 % de los casos.

Por el Teorema de los Números Primos la densidad cerca de x es $\approx 1/\ln x$; en el centro del rango ($x \approx 5 \times 10^{39}$) vale 1.0928 %, es decir **109.3 primos esperados** en 10 000 números, con desviación típica binomial $\sigma \approx 10{,}4$.

Cuadro 2. Primos hallados en 10 000 números de 40 cifras (semilla 2026)

Técnica	Primos	Densidad	z
LCG $(2^{61} - 1)$	106	1.060 %	$-0{,}32$
Mersenne Twister	116	1.160 %	$+0{,}65$
Hash SHA-256	87	0.870 %	$-2{,}14$
Blum-Blum-Shub	114	1.140 %	$+0{,}45$
Esperado (TNP)	109.3	1.093 %	—

Tres técnicas caen a menos de $0{,}7\sigma$ de lo esperado. SHA-256

queda a $-2{,}14\sigma$; con cuatro técnicas, un valor así aparece por azar con probabilidad cercana al 12 %, y para comprobar que no es un defecto del generador se repitió el experimento con las semillas 1 a 10: el promedio de primos fue 110.1 (LCG), 104.8 (MT), 109.4 (SHA-256) y 107.6 (BBS), todos compatibles con 109.3 (error típico del promedio $\approx 3{,}3$). Los primos hallados (423 en total) se guardan en `primos_encontrados.csv`. Ejemplos:

```
LCG   693304141368651403456574244046747215667 7
MT    529472536301641019034072267821768313808 3
SHA256 971954484777050200005938321124948422495 9
BBS   777460259028434665395921001094702211864 7
```

BBS es la técnica más lenta (1.10 s para 10 000 números, frente a $\leq 0{,}01$ s del resto) porque emite un bit por elevación al cuadrado modular; el test de primalidad de los 10 000 números toma $\approx 0{,}01$ s por técnica.

III. SISTEMA DE ENCRIPTACIÓN CON LOS PRIMOS ENCONTRADOS

Se construyó un RSA [4] cuyos factores son dos de los primos de 40 cifras hallados: $n = pq$, $\varphi = (p-1)(q-1)$, $e = 65537$, $d = e^{-1} \bmod \varphi$, con $c = m^e \bmod n$ y $m = c^d \bmod n$. Se exige que $p \neq q$, que ambos sean primos y que $\gcd(e, \varphi) = 1$.

A. *Cifrado por bloques*

Con n de 266 bits un mensaje cabe en un solo entero apenas si tiene 32 bytes, así que el sistema trocea el texto UTF-8 en bloques de 32 bytes. A cada bloque se le antepone el byte `0x01`, que conserva los ceros iniciales y garantiza $m < n$. Cada bloque se cifra por separado; el mensaje cifrado es la lista de enteros resultantes. También se pueden cifrar directamente números $0 \leq m < n$.

```
p (LCG) = 693304141368651403456574244046747215667 7
q (SHA256) = 971954484777050200005938321124948422495 9
n = p*q (80 cifras, 266 bits), e = 65537
Mensaje : 'HOLA RSA DESDE SISTEMAS COMPLEJOS'
Cifrado[0]: 42935951828314565587...460
Cifrado[1]: 39423169855720735276...989
Descifrado: igual al original -> True
```

IV. PRUEBAS DE ENCRIPTACIÓN

Se ejecutó una batería de 25 pruebas automáticas (Tabla 3); todas se superaron. Las pruebas P13 a P16 validan la máquina y los primos; P17 a P19 amplían las de cifrado, y P19 muestra el ataque de factorización contra un RSA pequeño. Los 30 sistemas RSA de la prueba P1 se formaron con los primeros 5 primos de cada una de las 6 combinaciones de técnicas distintas.

Cuadro 3. Resultados de las pruebas de encriptación

Id	Prueba	Res.
P1	$ed \equiv 1 \pmod{\varphi}$ en 30 pares (técnicas distintas)	30/30
P2	Ida y vuelta: texto corto, vacío, frase, Unicode (ñ, kanji, emoji), 2000 bytes, ceros iniciales	6/6
P3	Mismo mensaje con las 30 llaves	30/30
P4	204 números (0, 1, 2, $n-1$ y 200 aleatorios) y 10 primos de 40 cifras	214/214
P5	El cifrado no coincide con el texto plano	OK
P6	Llave privada incorrecta no recupera el mensaje	OK
P7	Alterar 1 bit del cifrado destruye el mensaje	OK
P8	Avalancha: 1 carácter cambia 139/265 bits (52.5 %)	OK
P9	Errores controlados: $m \geq n$, $m < 0$, $p = q$, p compuesto	4/4
P10	$E(m_1)E(m_2) = E(m_1 m_2)$ (homomorfismo)	OK
P11	Determinismo: mismo texto, mismo cifrado	OK
P12	100 kB: cifrar 0.03 s, descifrar 0.44 s (3125 bloques)	OK
P13	Reproducibilidad: misma semilla, misma salida; otra semilla, otra salida	OK
P14	La máquina respeta 10, 25, 40 y 60 cifras (4 técnicas $\times$ 100 números)	OK
P15	Dígitos uniformes: χ^2 máximo 18.25 $<$ 27,88 ($p = 0{,}001$, 9 g.l.)	OK
P16	Los 423 primos hallados pasan `sympy.isprime`	OK
P17	Ninguna de las otras 29 llaves descifra el mensaje	OK
P18	Intercambiar bloques cifrados altera el mensaje	OK
P19	Un n de $\sim$20 cifras se factoriza y se obtiene d	OK

Las pruebas P10 y P11 *confirman* dos debilidades conocidas del RSA de libro: es determinista (un atacante puede comprobar adivinanzas de texto) y es maleable (se pueden fabricar cifrados válidos de productos de mensajes). Los sistemas reales usan relleno aleatorio (OAEP) para evitarlo; aquí no se implementó.

A. *Limitaciones de seguridad*

Con n de 266 bits (80 cifras) el sistema es **didáctico, no seguro**: un número de ese tamaño se factoriza con criba del campo numérico en tiempo corto con un computador común, tras lo cual se obtiene d. Un RSA actual exige $n \geq 2048$ bits. El mismo programa serviría con primos de 300 cifras, pero no con `mt` ni `lcg` como fuente, ya que ambos son predecibles; para uso real se debe usar un CSPRNG (SHA-256, BBS con parámetros grandes o el generador del sistema operativo). El caso de OpenSSL en Debian [5], donde un generador debilitado dejó solo $\sim$32 768 claves posibles, ilustra el riesgo.

V. INTERFACES DE USO

El mismo programa ofrece tres formas de usar la máquina: (i) línea de comandos (`maquina`, `experimento`, `cifrar`, `pruebas`); (ii) un menú interactivo en consola (`menu`), que permite generar números, ejecutar el experimento, cifrar y descifrar mensajes con bloques pegados por el usuario y correr las pruebas;

y (iii) una interfaz gráfica en `tkinter` (`gui`) con cuatro pestañas: máquina PRNG, experimento (tabla de primos por técnica), cifrado/descifrado de mensajes y pruebas. La interfaz gráfica se probó de forma automática accionando cada botón: generó números, ejecutó el experimento, cifró y descifró un mensaje con Unicode y emoji, y reportó 25/25 pruebas superadas.

VI. CONCLUSIONES

Se construyó una máquina parametrizable con cuatro técnicas independientes que produce números pseudoaleatorios de 40 cifras; con ella se generaron 10 000 números por técnica y se hallaron 87 a 116 primos, consistentes con los 109.3 predichos por el Teorema de los Números Primos (confirmado con 10 semillas adicionales y con una segunda implementación de la primalidad). Con los 423 primos obtenidos se construyó un sistema RSA por bloques que superó las 25 pruebas de encriptación, incluyendo texto Unicode, mensajes largos, números y casos de fallo. El sistema demuestra el algoritmo de principio a fin, pero por el tamaño de sus llaves no protege datos reales.

REFERENCIAS

[1] D. E. Knuth, *The Art of Computer Programming, Vol. 2: Seminumerical Algorithms*, 3.ª ed. Addison-Wesley, 1997.

[2] S. K. Park y K. W. Miller, "Random number generators: good ones are hard to find," *Commun. ACM*, vol. 31, no. 10, pp. 1192–1201, 1988.

[3] L. Blum, M. Blum, y M. Shub, "A simple unpredictable pseudo-random number generator," *SIAM J. Comput.*, vol. 15, no. 2, pp. 364–383, 1986.

[4] R. L. Rivest, A. Shamir, y L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," *Commun. ACM*, vol. 21, no. 2, pp. 120–126, 1978.

[5] L. Weinmann, "Debian OpenSSL predictable random number generator," CVE-2008-0166, 2008.

APÉNDICE — CÓDIGO FUENTE (`generadores_primos.py`)

```python
import argparse
import csv
import hashlib
import math
import os
import random
import time

try:
    import gmpy2

    def es_primo(n):
        return bool(gmpy2.is_prime(n))
except ImportError:
    import sympy

    def es_primo(n):
        return sympy.isprime(n)

SEED = 2026
CIFRAS = 40
N_NUMEROS = 10_000
CSV_PRIMOS = "primos_encontrados.csv"


class GeneradorLCG:
    m = 2 ** 61 - 1
    a = 6364136223846793005 % m

    def __init__(self, seed):
        self.estado = (seed % (self.m - 1)) + 1

    def crudo(self, bits):
        v, n = 0, 0
        while n < bits:
            self.estado = (self.a * self.estado) % self.m
            v = (v << 61) | self.estado
            n += 61
        return v


class GeneradorMersenneTwister:

    def __init__(self, seed):
        self._rng = random.Random(seed)

    def crudo(self, bits):
        return self._rng.getrandbits(bits)


class GeneradorHashSHA256:

    def __init__(self, seed):
        self.semilla = str(seed).encode()
        self.contador = 0

    def crudo(self, bits):
        partes = []
        for _ in range((bits + 255) // 256):
            partes.append(hashlib.sha256(
                self.semilla + self.contador.to_bytes(8, "big")).digest())
            self.contador += 1
        return int.from_bytes(b"".join(partes), "big")


class GeneradorBBS:
    _P = None
    _Q = None

    @classmethod
    def _modulo(cls):
        if cls._P is None:
            def primo_3mod4(inicio):
                n = inicio | 3
                while not (n % 4 == 3 and es_primo(n)):
                    n += 4
                return n
            cls._P = primo_3mod4(2 ** 127 + 12345)
            cls._Q = primo_3mod4(2 ** 127 + 987654321)
        return cls._P * cls._Q
```

```
81
82      def __init__(self, seed):
83          M = self._modulo()
84          s = (seed % (M - 2)) + 2
85          while math.gcd(s, M) != 1:
86              s += 1
87          self.M = M
88          self.x = pow(s, 2, M)
89
90      def crudo(self, bits):
91          v = 0
92          for _ in range(bits):
93              self.x = pow(self.x, 2, self.M)
94              v = (v << 1) | (self.x & 1)
95          return v
96
97
98  TECNICAS = {
99      "lcg": ("LCG", GeneradorLCG),
100     "mt": ("Mersenne Twister", GeneradorMersenneTwister),
101     "sha256": ("Hash SHA-256", GeneradorHashSHA256),
102     "bbs": ("Blum-Blum-Shub", GeneradorBBS),
103 }
104
105
106 class MaquinaPRNG:
107
108     def __init__(self, tecnica="sha256", semilla=SEED, cifras=CIFRAS):
109         tecnica = tecnica.lower()
110         if tecnica not in TECNICAS:
111             raise ValueError(f"tecnica desconocida {tecnica!r}; "
112                              f"opciones: {', '.join(TECNICAS)}")
113         if cifras < 2:
114             raise ValueError("cifras debe ser >= 2")
115         self.tecnica = tecnica
116         self.nombre = TECNICAS[tecnica][0]
117         self.semilla = semilla
118         self.cifras = cifras
119         self.inferior = 10 ** (cifras - 1)
120         self.rango = 10 ** cifras - self.inferior
121         self.bits = self.rango.bit_length() + 64
122         self._gen = TECNICAS[tecnica][1](semilla)
123
124     def siguiente(self):
125         return self.inferior + (self._gen.crudo(self.bits) % self.rango)
126
127     def lote(self, k):
128         return [self.siguiente() for _ in range(k)]
129
130
131 def densidad_teorica(cifras=CIFRAS):
132     medio = (10 ** (cifras - 1) + 10 ** cifras) // 2
133     return 1 / math.log(medio)
134
135
136 def correr_experimento(cantidad=N_NUMEROS, semilla=SEED, cifras=CIFRAS,
137                        guardar_csv=True, verificar_sympy=True, verbose=True):
138     p_teo = densidad_teorica(cifras)
139     esperados = cantidad * p_teo
140     sigma = math.sqrt(cantidad * p_teo * (1 - p_teo))
141     if verbose:
142         print(f"Densidad teorica 1/ln(x) en el centro del rango: {p_teo*100:.4f}%")
143         print(f"Primos esperados en {cantidad} numeros: {esperados:.1f} "
144               f"(desv. tipica binomial ~{sigma:.1f})\n")
145
146     resultados = {}
147     for clave in TECNICAS:
148         maq = MaquinaPRNG(clave, semilla, cifras)
149         t0 = time.time()
150         numeros = maq.lote(cantidad)
151         t_gen = time.time() - t0
152         t0 = time.time()
153         primos = [(i, n) for i, n in enumerate(numeros) if es_primo(n)]
154         t_test = time.time() - t0
155
156         assert all(len(str(n)) == cifras for n in numeros), "numero sin 40 cifras"
157         repetidos = cantidad - len(set(numeros))
158         pares = sum(1 for n in numeros if n % 2 == 0)
159
160         coincide = None
161         if verificar_sympy:
162             import sympy
163             idx_primos = {i for i, _ in primos}
```

```
            muestra = [i for i in range(min(cantidad, 600)) if i not in idx_primos]
            ok_p = all(sympy.isprime(n) for _, n in primos)
            ok_c = not any(sympy.isprime(numeros[i]) for i in muestra)
            coincide = ok_p and ok_c

        r = {"nombre": maq.nombre, "primos": primos, "n": len(primos),
             "densidad": len(primos) / cantidad, "z": (len(primos) - esperados) / sigma,
             "t_gen": t_gen, "t_test": t_test, "repetidos": repetidos,
             "pct_pares": pares / cantidad * 100, "sympy_ok": coincide}
        resultados[clave] = r
        if verbose:
            print(f"=== {maq.nombre} ===")
            print(f" primos: {r['n']} / {cantidad} ({r['densidad']*100:.4f}%), "
                  f"z = {r['z']:+.2f}")
            print(f" repetidos: {repetidos}; pares: {r['pct_pares']:.2f}%; "
                  f"verificacion sympy: {coincide}")
            print(f" tiempo: generar {t_gen:.2f}s, primalidad {t_test:.2f}s")
            for _, n in primos[:3]:
                print(f" ej: {n}")
            print()

    if verbose:
        print("=== RESUMEN ===")
        print(f"{'Tecnica':<20}{'Primos':<8}{'Densidad %':<12}{'z':<8}")
        for r in resultados.values():
            print(f"{r['nombre']:<20}{r['n']:<8}{r['densidad']*100:<12.4f}{r['z']:+.2f}")

    if guardar_csv:
        with open(CSV_PRIMOS, "w", newline="", encoding="utf-8") as f:
            w = csv.writer(f)
            w.writerow(["tecnica", "indice", "primo"])
            for clave, r in resultados.items():
                for i, n in r["primos"]:
                    w.writerow([clave, i, n])
        if verbose:
            print(f"\nPrimos guardados en {CSV_PRIMOS}")
    return resultados


class SistemaRSA:
    E_PUBLICO = 65537

    def __init__(self, p, q):
        if p == q:
            raise ValueError("p y q deben ser primos distintos")
        if not (es_primo(p) and es_primo(q)):
            raise ValueError("p y q deben ser primos")
        self.p, self.q = p, q
        self.n = p * q
        self.phi = (p - 1) * (q - 1)
        if math.gcd(self.E_PUBLICO, self.phi) != 1:
            raise ValueError("e no es coprimo con phi(n); elegir otro par")
        self.e = self.E_PUBLICO
        self.d = pow(self.e, -1, self.phi)
        k = (self.n.bit_length() - 1) // 8
        self.tam_bloque = k - 1

    def clave_publica(self):
        return (self.e, self.n)

    def clave_privada(self):
        return (self.d, self.n)

    def claves_validas(self):
        return (self.e * self.d) % self.phi == 1

    def cifrar_entero(self, m):
        if not 0 <= m < self.n:
            raise ValueError("el entero debe cumplir 0 <= m < n")
        return pow(m, self.e, self.n)

    def descifrar_entero(self, c):
        return pow(c, self.d, self.n)

    def cifrar(self, texto):
        datos = texto.encode("utf-8")
        t = self.tam_bloque
        bloques = [datos[i:i + t] for i in range(0, len(datos), t)] or [b""]
        return [self.cifrar_entero(int.from_bytes(b"\x01" + b, "big")) for b in bloques]

    def descifrar(self, cifrado):
        datos = b""
        for c in cifrado:
```

```
            m = self.descifrar_entero(c)
            b = m.to_bytes((m.bit_length() + 7) // 8, "big")
            if not b or b[0] != 1:
                raise ValueError("bloque corrupto o clave incorrecta")
            datos += b[1:]
        return datos.decode("utf-8")

    @classmethod
    def desde_primos(cls, primos, intentos=50):
        for i in range(len(primos)):
            for j in range(i + 1, len(primos)):
                try:
                    return cls(primos[i], primos[j])
                except ValueError:
                    continue
        raise ValueError("no hay un par de primos valido")


def primos_por_tecnica(resultados):
    return {k: [n for _, n in r["primos"]] for k, r in resultados.items()}


def demostrar_rsa(resultados, mensaje="HOLA RSA DESDE SISTEMAS COMPLEJOS", verbose=True):
    pp = primos_por_tecnica(resultados)
    p, q = pp["lcg"][0], pp["sha256"][0]
    rsa = SistemaRSA(p, q)
    cif = rsa.cifrar(mensaje)
    des = rsa.descifrar(cif)
    if verbose:
        print("=== Sistema de encriptacion RSA con los primos encontrados ===")
        print(f"p (LCG) = {p}")
        print(f"q (SHA256) = {q}")
        print(f"n = p*q = {rsa.n} ({len(str(rsa.n))} cifras, {rsa.n.bit_length()} bits)")
        print(f"e = {rsa.e}\nd = {rsa.d}")
        print(f"e*d mod phi == 1 -> {rsa.claves_validas()}")
        print(f"Bloque: {rsa.tam_bloque} bytes de mensaje por bloque")
        print(f"Mensaje : {mensaje!r}")
        for i, c in enumerate(cif):
            print(f"Cifrado[{i}]: {c}")
        print(f"Descifrado: {des!r} -> coincide: {des == mensaje}")
    return rsa


def pruebas_encriptacion(resultados, verbose=True):
    pp = primos_por_tecnica(resultados)
    rng = random.Random(7)
    out = []

    def reg(nombre, ok, detalle=""):
        out.append((nombre, bool(ok), detalle))
        if verbose:
            print(f"[{'OK ' if ok else 'FALLA'}] {nombre} {detalle}")

    pares, validos = 0, 0
    sistemas = []
    claves = list(pp)
    for a in claves:
        for b in claves:
            if a <= b:
                continue
            for k in range(5):
                pares += 1
                try:
                    s = SistemaRSA(pp[a][k], pp[b][k])
                except ValueError:
                    continue
                if s.claves_validas():
                    validos += 1
                    sistemas.append(s)
    reg("P1 llaves: e*d = 1 (mod phi) en pares de tecnicas distintas",
        validos == pares and validos > 0, f"{validos}/{pares} pares")
    rsa = sistemas[0]

    textos = {
        "corto": "Hola",
        "vacio": "",
        "frase": "HOLA RSA DESDE SISTEMAS COMPLEJOS",
        "unicode": "\u00d1and\u00fa caf\u00e9 \u65e5\u672c\u8a9e \U0001F510 \u00bfqu\u00e9 tal?",
        "largo(2000 car.)": "Sistemas complejos. " * 100,
        "ceros iniciales": "\x00\x00abc\x00",
    }
    for nom, t in textos.items():
        c = rsa.cifrar(t)
```

```
        reg(f"P2 ida y vuelta texto {nom}", rsa.descifrar(c) == t,
            f"{len(t.encode('utf-8'))} bytes -> {len(c)} bloque(s)")

    msg = "Mensaje de prueba para todas las llaves"
    ok = all(s.descifrar(s.cifrar(msg)) == msg for s in sistemas)
    reg("P3 mismo mensaje con todas las llaves generadas", ok, f"{len(sistemas)} llaves")

    nums = [0, 1, 2, rsa.n - 1] + [rng.randrange(rsa.n) for _ in range(200)]
    ok = all(rsa.descifrar_entero(rsa.cifrar_entero(m)) == m for m in nums)
    reg("P4 ida y vuelta de numeros (bordes + 200 aleatorios)", ok, f"{len(nums)} numeros")
    n40 = [pp["mt"][i] for i in range(10)]
    ok = all(rsa.descifrar_entero(rsa.cifrar_entero(m)) == m for m in n40)
    reg("P4b numeros de 40 cifras (primos hallados) cifrados", ok, "10 numeros")

    c = rsa.cifrar("ATAQUE AL AMANECER")[0]
    reg("P5 el cifrado no es el texto plano ni su entero",
        c != int.from_bytes(b"\x01ATAQUE AL AMANECER", "big"))

    otra = sistemas[1]
    fallo = True
    try:
        fallo = otra.descifrar(rsa.cifrar("secreto importante")) == "secreto importante"
    except Exception:
        fallo = False
    reg("P6 clave privada equivocada NO recupera el mensaje", not fallo)

    c = rsa.cifrar("transferir 100 pesos")
    c2 = [c[0] ^ 1] + c[1:]
    try:
        r = rsa.descifrar(c2)
        det = r != "transferir 100 pesos"
    except Exception:
        det = True
    reg("P7 alterar 1 bit del cifrado destruye el mensaje", det)

    a = rsa.cifrar("A" * 20)[0]
    b = rsa.cifrar("A" * 19 + "B")[0]
    ham = bin(a ^ b).count("1")
    reg("P8 cambiar 1 caracter cambia ~la mitad de los bits del bloque",
        0.3 < ham / rsa.n.bit_length() < 0.7,
        f"{ham}/{rsa.n.bit_length()} bits = {ham/rsa.n.bit_length()*100:.1f}%")

    errs = 0
    for f in (lambda: rsa.cifrar_entero(rsa.n), lambda: rsa.cifrar_entero(-1),
              lambda: SistemaRSA(pp["lcg"][0], pp["lcg"][0]),
              lambda: SistemaRSA(pp["lcg"][0] + 1, pp["mt"][0])):
        try:
            f()
        except ValueError:
            errs += 1
    reg("P9 errores controlados (m>=n, m<0, p==q, p compuesto)", errs == 4, f"{errs}/4")

    m1, m2 = 123456789, 987654321
    ok = (rsa.cifrar_entero(m1) * rsa.cifrar_entero(m2)) % rsa.n == rsa.cifrar_entero(m1 * m2)
    reg("P10 propiedad homomorfica E(m1)E(m2)=E(m1*m2) (debilidad del RSA de libro)", ok)

    reg("P11 RSA de libro es determinista (mismo texto -> mismo cifrado)",
        rsa.cifrar("igual") == rsa.cifrar("igual"))

    t0 = time.time()
    texto = "x" * 100_000
    c = rsa.cifrar(texto)
    t1 = time.time()
    d = rsa.descifrar(c)
    t2 = time.time()
    reg("P12 rendimiento 100 kB", d == texto,
        f"cifrar {t1-t0:.2f}s, descifrar {t2-t1:.2f}s, {len(c)} bloques")

    ok = True
    for k in TECNICAS:
        a = MaquinaPRNG(k, 99).lote(20)
        b = MaquinaPRNG(k, 99).lote(20)
        c2 = MaquinaPRNG(k, 100).lote(20)
        ok &= (a == b) and (a != c2)
    reg("P13 reproducibilidad: misma semilla igual salida, otra semilla distinta", ok,
        "4 tecnicas")

    ok = all(len(str(n)) == c
             for k in TECNICAS for c in (10, 25, 40, 60)
             for n in MaquinaPRNG(k, 5, c).lote(100))
    reg("P14 la maquina respeta cifras = 10, 25, 40, 60", ok, "4 tecnicas x 4 tamanos x 100")

    peor = 0.0
```

```
    for k in TECNICAS:
        cont = [0] * 10
        for n in MaquinaPRNG(k, SEED).lote(2000):
            for ch in str(n)[1:]:
                cont[int(ch)] += 1
        tot = sum(cont)
        chi2 = sum((o - tot / 10) ** 2 / (tot / 10) for o in cont)
        peor = max(peor, chi2)
    reg("P15 digitos uniformes (chi2 < 27.88 en las 4 tecnicas)", peor < 27.88,
        f"peor chi2 = {peor:.2f}")

    try:
        import sympy
        todos = [n for lst in pp.values() for n in lst]
        reg("P16 todos los primos hallados pasan sympy.isprime",
            all(sympy.isprime(n) for n in todos), f"{len(todos)} primos")
    except ImportError:
        reg("P16 verificacion con sympy", True, "omitida (sympy no instalado)")

    msg = "mensaje confidencial"
    c = rsa.cifrar(msg)
    fallos = 0
    for s2 in sistemas[1:]:
        try:
            if s2.descifrar(c) == msg:
                fallos += 1
        except Exception:
            pass
    reg("P17 ninguna de las otras llaves descifra el mensaje", fallos == 0,
        f"{len(sistemas)-1} llaves probadas")

    texto = "A" * 32 + "B" * 32
    cb = rsa.cifrar(texto)
    reg("P18 intercambiar bloques cifrados cambia el mensaje",
        rsa.descifrar([cb[1], cb[0]]) != texto and len(cb) == 2)

    try:
        import sympy
        pq = MaquinaPRNG("sha256", 3, 10)
        pa = next(n for n in pq.lote(5000) if es_primo(n))
        qa = next(n for n in pq.lote(5000) if es_primo(n) and n != pa)
        chico = SistemaRSA(pa, qa)
        t0 = time.time()
        fac = sorted(sympy.factorint(chico.n))
        d_rec = pow(chico.e, -1, (fac[0] - 1) * (fac[1] - 1))
        c_ = chico.cifrar_entero(123456)
        reg("P19 un n de ~20 cifras se factoriza y se rompe la llave",
            pow(c_, d_rec, chico.n) == 123456,
            f"{time.time()-t0:.2f}s; con 40+40 cifras hace falta un algoritmo mucho mas pesado (NFS)")
    except (ImportError, StopIteration, ValueError):
        reg("P19 ataque de factorizacion", True, "omitida")

    n_ok = sum(1 for _, ok, _ in out if ok)
    if verbose:
        print(f"\nResultado: {n_ok}/{len(out)} pruebas superadas")
    return out


def sistema_desde_resultados(resultados, tec_p="lcg", tec_q="sha256"):
    pp = primos_por_tecnica(resultados)
    if tec_p == tec_q:
        return SistemaRSA(pp[tec_p][0], pp[tec_p][1])
    return SistemaRSA(pp[tec_p][0], pp[tec_q][0])


def _pedir(texto, defecto, tipo=str):
    try:
        v = input(f"{texto} [{defecto}]: ").strip()
    except EOFError:
        return defecto
    if not v:
        return defecto
    try:
        return tipo(v)
    except ValueError:
        print(" valor no valido, se usa el defecto")
        return defecto


def menu_interactivo():
    resultados = None
    rsa = None
    while True:
```

```
        print("\n===== MAQUINA DE ENCRIPTACION CON PRIMOS GRANDES =====")
        print(" 1) Maquina PRNG: generar numeros de 40 cifras")
        print(" 2) Experimento: 10 000 numeros por tecnica y primos hallados")
        print(" 3) Cifrar un mensaje")
        print(" 4) Descifrar un mensaje (pega los bloques separados por comas)")
        print(" 5) Pruebas de encriptacion")
        print(" 6) Salir")
        try:
            op = input("Opcion: ").strip()
        except EOFError:
            return
        if op == "1":
            t = _pedir(f"Tecnica ({'/'.join(TECNICAS)})", "sha256")
            if t not in TECNICAS:
                print(" tecnica desconocida"); continue
            sem = _pedir("Semilla", SEED, int)
            cif = _pedir("Cifras", CIFRAS, int)
            cant = _pedir("Cantidad", 5, int)
            try:
                for n in MaquinaPRNG(t, sem, cif).lote(cant):
                    print(f" {n} ({'PRIMO' if es_primo(n) else 'compuesto'})")
            except ValueError as ex:
                print(" error:", ex)
        elif op == "2":
            cant = _pedir("Cantidad por tecnica", N_NUMEROS, int)
            resultados = correr_experimento(cant, SEED)
            rsa = None
        elif op in ("3", "4"):
            if resultados is None:
                print("(generando primos con el experimento...)")
                resultados = correr_experimento(verbose=False)
            if rsa is None:
                tp = _pedir("Tecnica del primo p", "lcg")
                tq = _pedir("Tecnica del primo q", "sha256")
                try:
                    rsa = sistema_desde_resultados(resultados, tp, tq)
                except (KeyError, ValueError) as ex:
                    print(" no se pudo crear la llave:", ex); continue
                print(f" p = {rsa.p}\n q = {rsa.q}\n n = {rsa.n}\n e = {rsa.e}")
            if op == "3":
                msg = _pedir("Mensaje", "Hola mundo")
                cif = rsa.cifrar(msg)
                print("Cifrado (bloques separados por comas):")
                print(",".join(str(c) for c in cif))
            else:
                try:
                    txt = input("Bloques: ")
                    cif = [int(x) for x in txt.replace(" ", "").split(",") if x]
                    print("Mensaje:", rsa.descifrar(cif))
                except (ValueError, EOFError) as ex:
                    print(" entrada o llave invalida:", ex)
        elif op == "5":
            if resultados is None:
                resultados = correr_experimento(verbose=False)
            pruebas_encriptacion(resultados)
        elif op == "6":
            return
        else:
            print(" opcion no valida")


def interfaz_grafica():
    try:
        import tkinter as tk
        from tkinter import ttk, messagebox
    except ImportError:
        print("tkinter no esta instalado en este Python (en Linux: "
              "sudo apt install python3-tk). Usa el modo 'menu'.")
        return

    estado = {"res": None, "rsa": None}
    root = tk.Tk()
    root.title("Maquina de encriptacion con primos grandes")
    root.geometry("900x620")
    nb = ttk.Notebook(root)
    nb.pack(fill="both", expand=True, padx=6, pady=6)

    def texto(parent, alto):
        t = tk.Text(parent, height=alto, wrap="word", font=("Courier", 10))
        t.pack(fill="both", expand=True, padx=6, pady=4)
        return t

    def escribir(t, s, borrar=True):
```

```
        if borrar:
            t.delete("1.0", "end")
        t.insert("end", s)

    def resultados():
        if estado["res"] is None:
            root.config(cursor="watch"); root.update()
            estado["res"] = correr_experimento(verbose=False)
            root.config(cursor="")
        return estado["res"]

    f1 = ttk.Frame(nb); nb.add(f1, text="1. Maquina PRNG")
    fila = ttk.Frame(f1); fila.pack(fill="x", padx=6, pady=6)
    v_tec = tk.StringVar(value="sha256")
    v_sem = tk.StringVar(value=str(SEED))
    v_cif = tk.StringVar(value=str(CIFRAS))
    v_can = tk.StringVar(value="5")
    for lbl, w in (("Tecnica", ttk.Combobox(fila, textvariable=v_tec, values=list(TECNICAS),
                                            state="readonly", width=8)),
                   ("Semilla", ttk.Entry(fila, textvariable=v_sem, width=10)),
                   ("Cifras", ttk.Spinbox(fila, from_=2, to=200, textvariable=v_cif, width=5)),
                   ("Cantidad", ttk.Spinbox(fila, from_=1, to=1000, textvariable=v_can, width=6))):
        ttk.Label(fila, text=lbl).pack(side="left", padx=(8, 2)); w.pack(side="left")
    out1 = texto(f1, 20)

    def generar():
        try:
            maq = MaquinaPRNG(v_tec.get(), int(v_sem.get()), int(v_cif.get()))
            lineas = [f"{n} ({len(str(n))} cifras, {'PRIMO' if es_primo(n) else 'compuesto'})"
                      for n in maq.lote(int(v_can.get()))]
            escribir(out1, f"Tecnica: {maq.nombre}\n\n" + "\n".join(lineas))
        except ValueError as ex:
            messagebox.showerror("Parametros invalidos", str(ex))
    ttk.Button(fila, text="Generar", command=generar).pack(side="left", padx=12)

    f2 = ttk.Frame(nb); nb.add(f2, text="2. Experimento")
    fila = ttk.Frame(f2); fila.pack(fill="x", padx=6, pady=6)
    v_n = tk.StringVar(value=str(N_NUMEROS))
    ttk.Label(fila, text="Numeros por tecnica").pack(side="left")
    ttk.Entry(fila, textvariable=v_n, width=8).pack(side="left", padx=4)
    tabla = ttk.Treeview(f2, columns=("t", "p", "d", "z"), show="headings", height=5)
    for c, h in (("t", "Tecnica"), ("p", "Primos"), ("d", "Densidad %"), ("z", "z")):
        tabla.heading(c, text=h); tabla.column(c, width=120)
    tabla.pack(fill="x", padx=6)
    out2 = texto(f2, 14)

    def experimento():
        try:
            cant = int(v_n.get())
        except ValueError:
            messagebox.showerror("Error", "cantidad no valida"); return
        root.config(cursor="watch"); root.update()
        estado["res"] = correr_experimento(cant, SEED, verbose=False)
        estado["rsa"] = None
        root.config(cursor="")
        tabla.delete(*tabla.get_children())
        txt = f"Esperados ~{cant*densidad_teorica():.1f} primos por tecnica.\n\n"
        for k, r in estado["res"].items():
            tabla.insert("", "end", values=(r["nombre"], r["n"],
                                            f"{r['densidad']*100:.4f}", f"{r['z']:+.2f}"))
            txt += f"{r['nombre']}: primeros primos\n"
            txt += "".join(f" {n}\n" for _, n in r["primos"][:3])
        escribir(out2, txt + f"\nPrimos guardados en {CSV_PRIMOS}")
    ttk.Button(fila, text="Ejecutar experimento", command=experimento).pack(side="left", padx=8)

    f3 = ttk.Frame(nb); nb.add(f3, text="3. Cifrado")
    fila = ttk.Frame(f3); fila.pack(fill="x", padx=6, pady=6)
    v_p = tk.StringVar(value="lcg"); v_q = tk.StringVar(value="sha256")
    ttk.Label(fila, text="Primo p de").pack(side="left")
    ttk.Combobox(fila, textvariable=v_p, values=list(TECNICAS), state="readonly",
                 width=8).pack(side="left", padx=4)
    ttk.Label(fila, text="Primo q de").pack(side="left")
    ttk.Combobox(fila, textvariable=v_q, values=list(TECNICAS), state="readonly",
                 width=8).pack(side="left", padx=4)
    info = texto(f3, 6)
    ttk.Label(f3, text="Mensaje:").pack(anchor="w", padx=6)
    t_msg = texto(f3, 4)
    ttk.Label(f3, text="Cifrado (bloques separados por comas):").pack(anchor="w", padx=6)
    t_cif = texto(f3, 6)
    ttk.Label(f3, text="Descifrado:").pack(anchor="w", padx=6)
    t_des = texto(f3, 3)

    def crear():
```

```
        try:
            estado["rsa"] = sistema_desde_resultados(resultados(), v_p.get(), v_q.get())
        except ValueError as ex:
            messagebox.showerror("Llave invalida", str(ex)); return None
        r = estado["rsa"]
        escribir(info, f"p = {r.p}\nq = {r.q}\nn = {r.n} ({r.n.bit_length()} bits)\n"
                       f"e = {r.e}\nd = {r.d}\nbloque = {r.tam_bloque} bytes")
        return r

    def cifrar():
        r = crear()
        if r:
            msg = t_msg.get("1.0", "end").rstrip("\n")
            escribir(t_cif, ",".join(str(c) for c in r.cifrar(msg)))
            escribir(t_des, "")

    def descifrar():
        r = estado["rsa"] or crear()
        if not r:
            return
        try:
            bloques = [int(x) for x in t_cif.get("1.0", "end").replace("\n", "")
                       .replace(" ", "").split(",") if x]
            escribir(t_des, r.descifrar(bloques))
        except ValueError as ex:
            messagebox.showerror("No se pudo descifrar", str(ex))
    fb = ttk.Frame(f3); fb.pack(fill="x", padx=6, pady=4)
    ttk.Button(fb, text="Cifrar (crea llaves nuevas)", command=cifrar).pack(side="left")
    ttk.Button(fb, text="Descifrar", command=descifrar).pack(side="left", padx=8)

    f4 = ttk.Frame(nb); nb.add(f4, text="4. Pruebas")
    out4 = texto(f4, 25)

    def pruebas():
        root.config(cursor="watch"); root.update()
        res = pruebas_encriptacion(resultados(), verbose=False)
        root.config(cursor="")
        lineas = [f"[{'OK ' if ok else 'FALLA'}] {n} {d}" for n, ok, d in res]
        lineas.append(f"\nResultado: {sum(ok for _, ok, _ in res)}/{len(res)} pruebas superadas")
        escribir(out4, "\n".join(lineas))
    ttk.Button(f4, text="Ejecutar pruebas", command=pruebas).pack(anchor="w", padx=6, pady=6)
    out4.pack_forget(); out4.pack(fill="both", expand=True, padx=6, pady=4)

    root.mainloop()


def main():
    ap = argparse.ArgumentParser(description="Maquina para encriptar mensajes con numeros primos grandes")
    sub = ap.add_subparsers(dest="cmd")
    m = sub.add_parser("maquina", help="usar la maquina PRNG")
    m.add_argument("--tecnica", default="sha256", choices=list(TECNICAS))
    m.add_argument("--semilla", type=int, default=SEED)
    m.add_argument("--cifras", type=int, default=CIFRAS)
    m.add_argument("--cantidad", type=int, default=5)
    e = sub.add_parser("experimento", help="generar N numeros por tecnica y hallar primos")
    e.add_argument("--cantidad", type=int, default=N_NUMEROS)
    e.add_argument("--semilla", type=int, default=SEED)
    c = sub.add_parser("cifrar", help="cifrar/descifrar un mensaje con los primos hallados")
    c.add_argument("--mensaje", default="HOLA RSA DESDE SISTEMAS COMPLEJOS")
    sub.add_parser("pruebas", help="bateria de pruebas de encriptacion")
    sub.add_parser("menu", help="menu interactivo en consola")
    sub.add_parser("gui", help="interfaz grafica (tkinter)")
    sub.add_parser("todo", help="demo completa (por defecto)")
    args = ap.parse_args()
    cmd = args.cmd or "todo"

    if cmd == "menu":
        menu_interactivo()
        return
    if cmd == "gui":
        interfaz_grafica()
        return

    if cmd == "maquina":
        maq = MaquinaPRNG(args.tecnica, args.semilla, args.cifras)
        print(f"Maquina: tecnica={maq.nombre}, semilla={args.semilla}, cifras={args.cifras}")
        for n in maq.lote(args.cantidad):
            print(f"{n} ({len(str(n))} cifras, {'PRIMO' if es_primo(n) else 'compuesto'})")
        return

    if cmd == "todo":
        print("### 1. LA MAQUINA: una salida de 40 cifras por tecnica (semilla=42)")
        for k in TECNICAS:
```

```
            n = MaquinaPRNG(k, 42).siguiente()
            print(f" {TECNICAS[k][0]:<18}{n} ({len(str(n))} cifras)")
        print("\n### 2. EXPERIMENTO: 10 000 numeros de 40 cifras por tecnica")
    cant = getattr(args, "cantidad", N_NUMEROS) if cmd == "experimento" else N_NUMEROS
    sem = getattr(args, "semilla", SEED) if cmd == "experimento" else SEED
    res = correr_experimento(cant, sem, verbose=cmd in ("experimento", "todo"))
    if cmd == "experimento":
        return
    if cmd in ("cifrar", "todo"):
        print("\n### 3. SISTEMA DE ENCRIPTACION")
        demostrar_rsa(res, getattr(args, "mensaje", "HOLA RSA DESDE SISTEMAS COMPLEJOS"))
    if cmd in ("pruebas", "todo"):
        print("\n### 4. PRUEBAS DE ENCRIPTACION")
        pruebas_encriptacion(res)


if __name__ == "__main__":
    main()
```